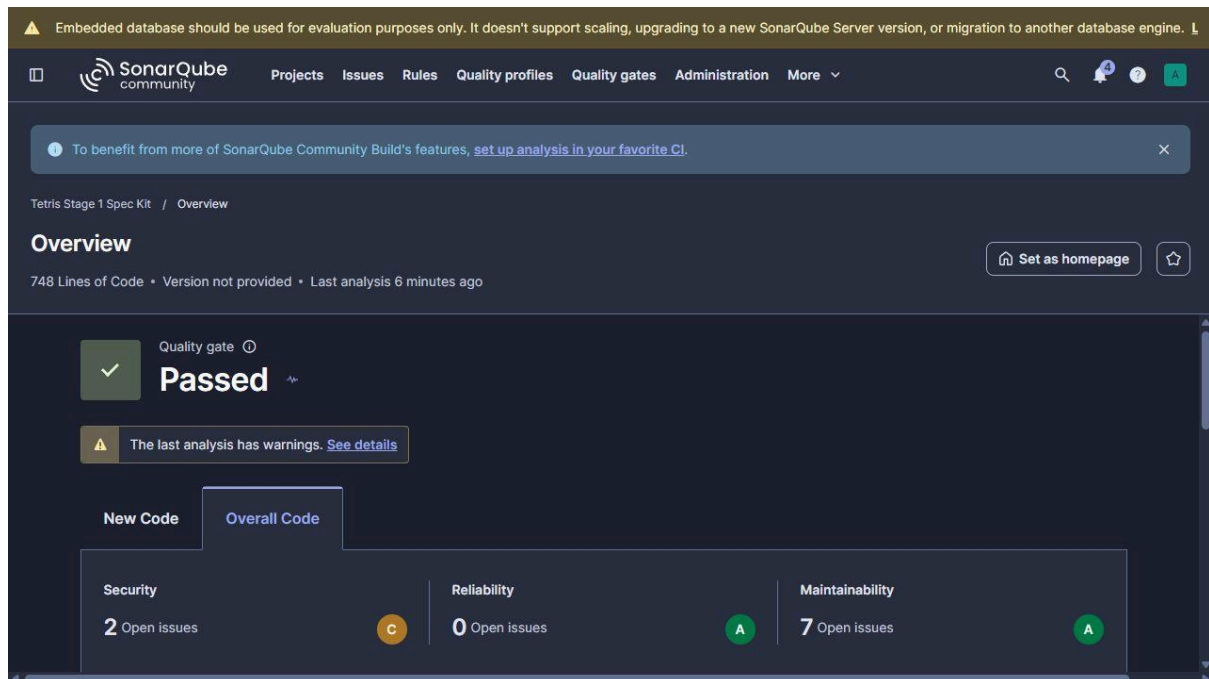


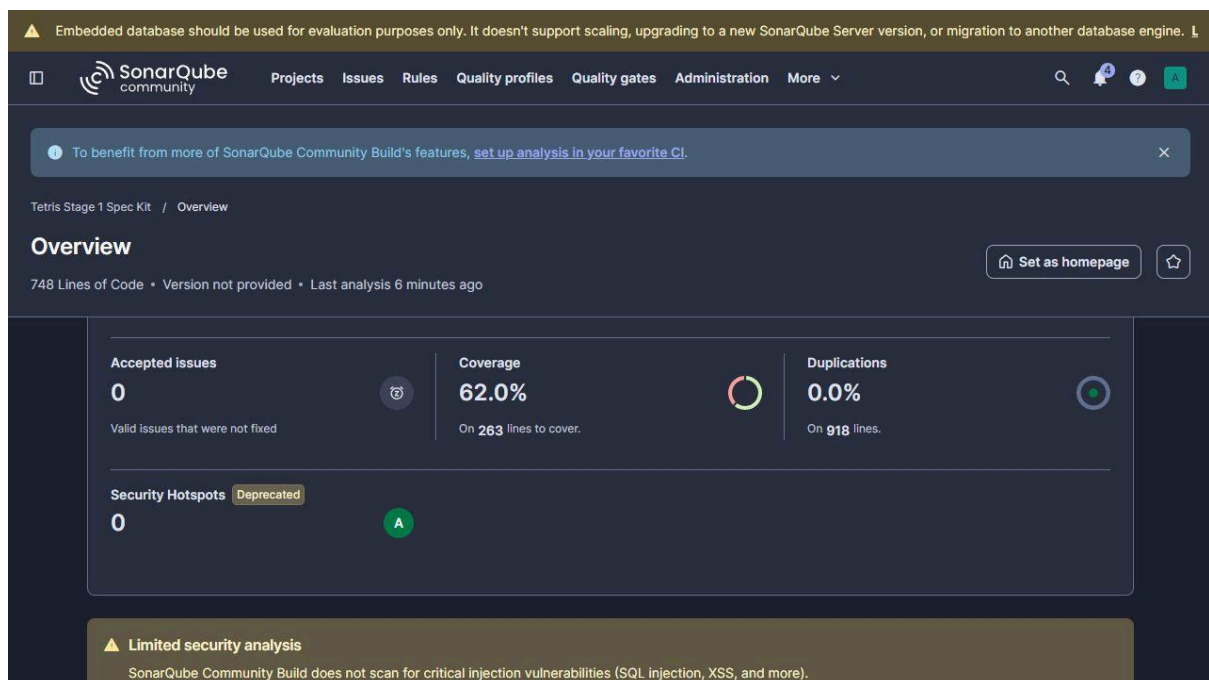
Tetris — Etapa 1, opción A

Comparación experimental de OpenSpec, GitHub Spec Kit, Tessl, BMad Method y Kiro

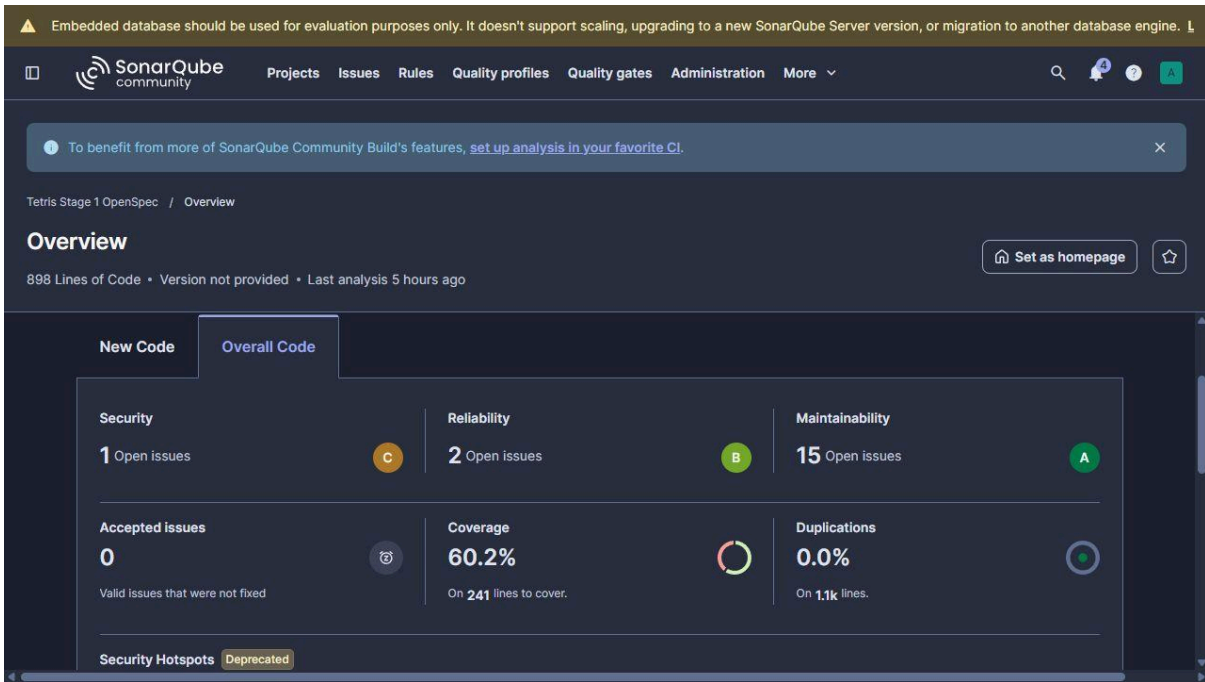
Evidencia SonarQube: Spec Kit (código global, 29 de septiembre de 2026).



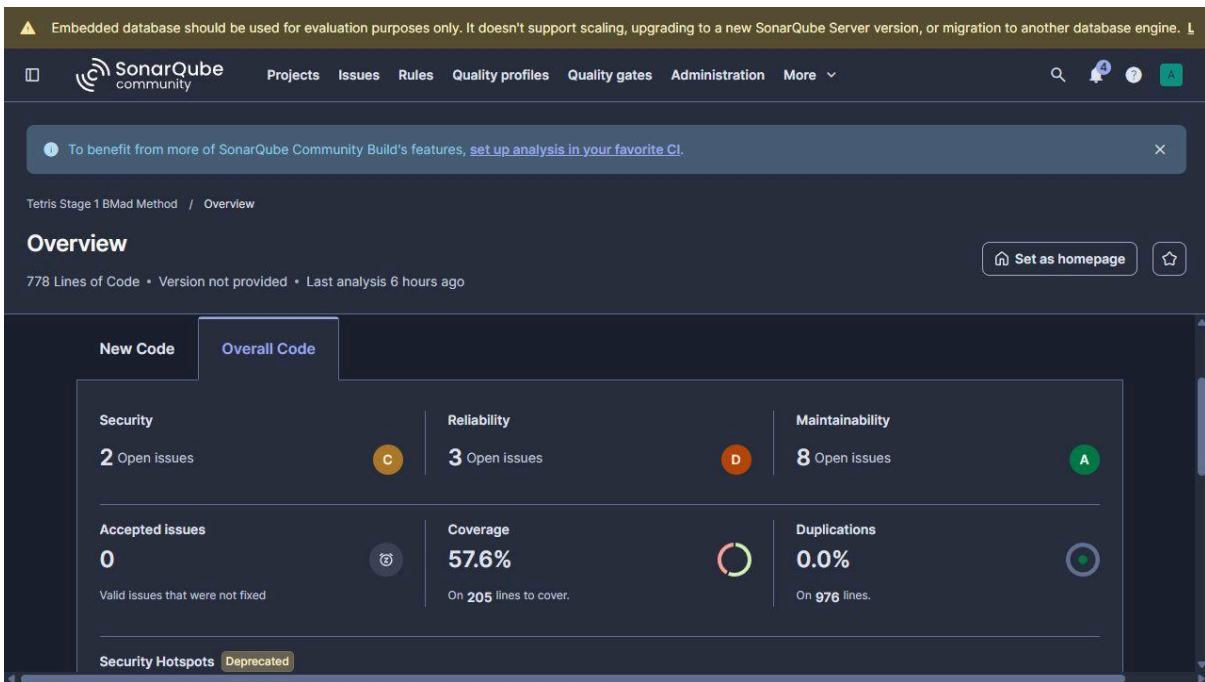
Calidad y hallazgos de Spec Kit (misma vista de código global).



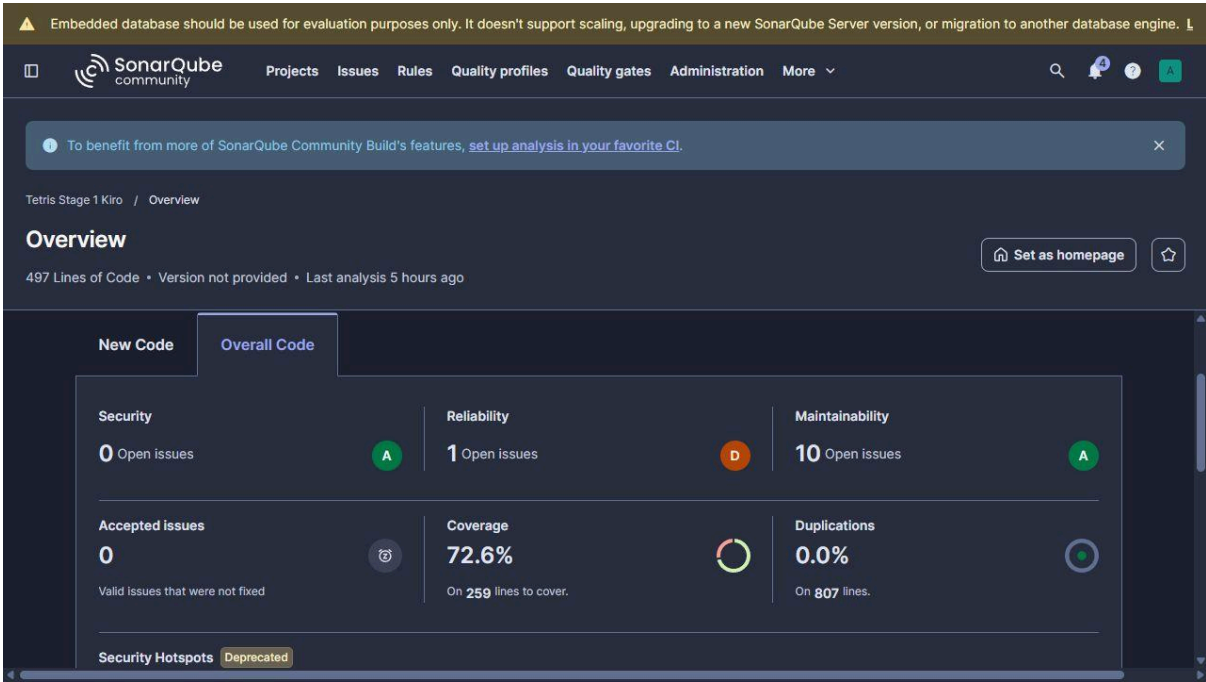
OpenSpec — panel SonarQube, código global; 898 líneas de código, 60,2 % de cobertura, 0,0 % de duplicación.



BMad Method — panel SonarQube, código global; 778 líneas de código, 57,6 % de cobertura, 0,0 % de duplicación.



Kiro — panel SonarQube, código global; 497 líneas de código, 72,6 % de cobertura, 0,0 % de duplicación.



Matriz comparativa

Alcance: únicamente Tetris Etapa 1, opción A. Las cinco ejecuciones recibieron la misma semilla inicial (SHA-256: 2AE084C9C64FC41F65D0E07581CED9E30B6C68891D9F8688A37F34594ECFD1C5). El juego usa una cuadrícula de 10 × 20, siete tetrominós sin rotación, desplazamiento lateral y descenso suave, gravedad de 500 ms, bloqueo, eliminación de líneas, fin de partida y reinicio. El backend FastAPI conserva el estado en memoria y el frontend usa React con TypeScript. No se hicieron commits.

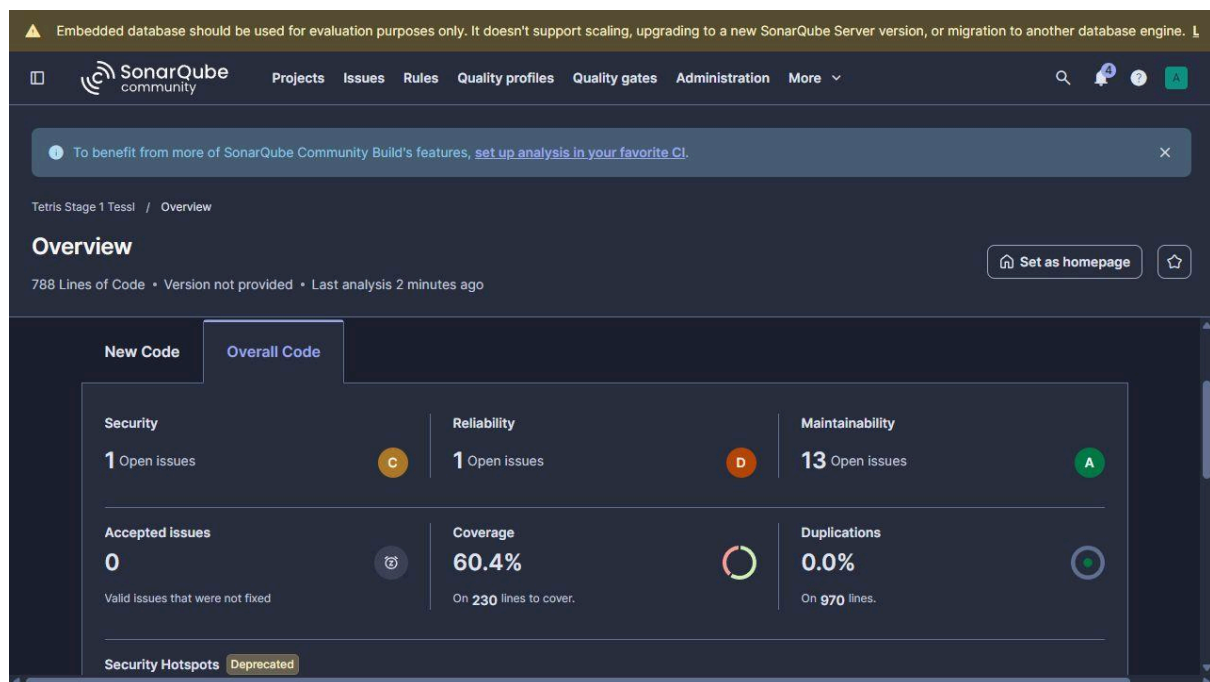
Protocolo: OpenSpec, Spec Kit, Tessl y BMad se ejecutaron con Gemini CLI 0.60.0, modelo Gemini 3.5 Flash mediante Vertex AI en el proyecto GCP acordado. Kiro CLI 2.25.0 se ejecutó en modo de especificación con selección automática de modelo y cuenta GitHub. Cada herramienta tiene artefactos separados. SonarQube Community Build 26.9 analizó backend y frontend; solo se importó la cobertura de pruebas Python. Por ello la cobertura global del panel es inferior a la cobertura de backend reportada por pytest.

Las líneas y caracteres de especificación y código se cuentan desde los archivos finales. La densidad es líneas físicas de especificación por archivo. Los tokens de Gemini corresponden al tráfico de modelos registrado por CLI, con reenvío de contexto y subagentes: no equivalen a cobro ni a tokens únicos de la semilla. El registro ACP de Kiro no expone tokens de entrada/salida verificables, por lo que se indica N/D.

Herramienta	Requisitos y especificación	Código (L; C)	Pruebas y Sonar
OpenSpec	30 req; 442 L; 23.969 C; 110,5 L/archivo	1.366 L; 39.957 C	13 backend; 91% backend; 60,2% Sonar; 898 ncloc; 15

			smells; 1 vul.
Spec Kit	38 req; 1.002 L; 56.433 C; 125,25 L/archivo	1.412 L; 41.297 C	18 backend + 5 frontend; 91% backend; 62,0% Sonar; 748 ncloc; 7 smells; 2 vul.
BMad Method	30 req; 402 L; 25.637 C; 100,5 L/archivo	1.210 L; 35.034 C	7 backend; 91% backend; 57,6% Sonar; 778 ncloc; 8 smells; 2 vul.
Kiro	35 req; 608 L; 28.780 C; 202,67 L/archivo	1.207 L; 33.641 C	39 backend; 99% backend; 72,6% Sonar; 497 ncloc; 10 smells; 0 vul.
Tessl	34 req; 136 L; 10.093 C; 68,0 L/archivo	1.299 L; 37.962 C	9 backend + 2 frontend; 91% backend; 60,4% Sonar; 788 ncloc; 13 smells; 1 vul.

Evidencia Tessl — panel SonarQube, código global; 788 líneas de código, 60,4 % de cobertura, 0,0 % de duplicación.



Proceso y uso de modelos

Tiempo = suma de ejecuciones CLI registradas, sin instalación, pruebas independientes ni análisis Sonar. Prompts = invocaciones CLI documentadas; incluyen etapas normales del método y correcciones. Tokens = entrada/salida observada en salidas JSON y

transcripciones conservadas; «≥» indica un mínimo porque faltan sesiones antiguas. Kiro: N/D por falta de contadores verificables en ACP.

Herramienta	Prompts	Tiempo CLI (min)	Entrada observada	Salida observada
OpenSpec	6	51,1	≥5.816.534	≥39.707
Spec Kit	6	56,0	≥14.171.471	≥90.198
BMad Method	2	19,1	4.443.575	34.646
Tessl	8	81,7	≥5.952.918	≥41.324
Kiro	1	9,1	N/D	N/D

Disponibilidad por sesión: el CSV de evidencia registra 1 sesión sin tokens de OpenSpec, 1 de Spec Kit y 3 de Tessl; BMad tiene los dos registros y Kiro no expone estos contadores. Los mínimos anteriores se construyeron con estadísticas JSON del CLI y, donde faltaban, con JSONL conservado. La comparación de eficiencia de tokens queda limitada por esas ausencias y por el modelo distinto de Kiro.

Intervenciones posteriores a la semilla: OpenSpec 5, Spec Kit 5, Tessl 7, BMad 1 y Kiro 0. Este recuento incluye pasos normales del flujo de cada método y reintentos; no mide por sí solo errores. Entre las correcciones verificadas estuvieron la sintaxis nativa de OpenSpec, un escenario imposible y fallas de build/importación de Spec Kit, el alcance de controles de Tessl y la atomización de requisitos de BMad. Las materializaciones manuales de texto generado para OpenSpec y Spec Kit se registraron como intervenciones; el código de aplicación se corrigió mediante las especificaciones y sus herramientas.

Indicadores complementarios de SonarQube

Herramienta	Gate / dup.	Deuda (min)	Comp. cogn.	S/R/M abiertas
OpenSpec	OK / 0,0%	81	76	1 / 2 / 15
Spec Kit	OK / 0,0%	35	78	2 / 0 / 7
BMad Method	OK / 0,0%	45	64	2 / 3 / 8
Tessl	OK / 0,0%	71	69	1 / 1 / 13
Kiro	OK / 0,0%	38	45	0 / 1 / 10

S/R/M = hallazgos abiertos con impacto en seguridad, confiabilidad y mantenibilidad que muestra el panel. La API clásica informó 0 bugs en las cinco variantes; sus categorías no son idénticas a los impactos S/R/M. La puerta de calidad predeterminada figura en OK para las cinco, aunque el código global conserva hallazgos. Las vulnerabilidades de OpenSpec, Spec Kit, Tessl y BMad son la regla python:S2245 por selección pseudoaleatoria de piezas; su relevancia para este juego requiere revisión contextual. SonarQube Community Build

limita el análisis de inyecciones y no mide escalabilidad; complejidad y duplicación son solo indicadores indirectos.

Lectura comparativa: Kiro terminó con una invocación CLI registrada, el menor tiempo (9,1 min), la cobertura global más alta (72,6 %) y ningún hallazgo de seguridad abierto. Su modelo automático y la ausencia de tokens verificables impiden una comparación de eficiencia de modelo en igualdad de condiciones. BMad requirió menos tiempo CLI que las otras variantes con Gemini (19,1 min), pero tuvo la menor cobertura global (57,6 %) y dos hallazgos de seguridad. Spec Kit produjo la especificación más extensa y cinco pruebas de frontend. Tessl requirió más reintentos y correcciones, en parte por respuestas de planificación y errores 429 de Vertex AI. OpenSpec validó sus cuatro artefactos nativos y completó 24 tareas; necesitó materializar el texto generado y corregir su sintaxis. Estas son observaciones de una ejecución por herramienta, no una estimación estadística de rendimiento general.

Evidencia reproducible: en cada carpeta de herramienta, docs/ conserva prompts, salidas, tiempos, pruebas, cobertura y archivos sonar-*.json. El archivo shared/metrics-preliminary.json contiene las líneas y caracteres finales; shared/token-sessions.csv detalla los tokens disponibles por sesión; PROGRESS.md documenta decisiones, comandos, intervenciones y siguientes pasos.